Unicode in the Library, Part 1: UTF Transcoding

Document #:	P2728R5
Date:	2023-07-05
Project:	Programming Language C++
Audience:	SG-16 Unicode LEWG

Reply-to:	Zach Laine <whatwasthataddress@gmail.com>

Contents

§ 1 Changelog

§ 1.1 Changes since R0

- When naming code points in interfaces, use `char32_t`.
- When naming code units in interfaces, use `charN_t`.
- Remove each eager algorithm, leaving in its corresponding view.
- Remove all the output iterators.
- Change template parameters to `utfN_view` to the types of the from-range, instead of the types of the transcoding iterators used to implement the view.
- Remove all make-functions.
- Replace the misbegotten `as_utfN()` functions with the `as_utfN` view adaptors that should have been there all along.
- Add missing `transcoding_error_handler` concept.
- Turn `unpack_iterator_and_sentinel` into a CPO.
- Lower the UTF iterator concepts from bidirectional to input.

§ 1.2 Changes since R1

- Reintroduce the transcoding-from-a-buffer example.
- Generalize `null_sentinel_t` to a non-Unicode-specific facility.

— In utility functions that search for ill-formed encoding, take a range argument instead of a pair of iterator arguments.

— Replace `utf{8,16,32}_view` with a single `utf_view`.

1.3 Changes since R2

— Add `noexcept` where appropriate.

— Remove non-essential constants and utility functions, and elaborate on the usage of the ones that remain.

— Note differences from similar elements proposed in [P1629R1].

— Extend the examples slightly.

— Correct an error in the description of the view adaptors' semantics, and provide several examples of their use.

1.4 Changes since R3

— Changed the definition of the `code_unit` concept, and added `as_charN_t` adaptors.

— Removed the utility functions and Unicode-related constants, except `replacement_character`.

— Changed the constraint on `utf_iterator` slightly.

— Change `null_sentinel_t` back to being Unicode-specific.

1.5 Changes since R4

— Replace `unpacking_owning_view` with `unpacking_view`, and use it to do unpacking, rather than sometimes doing the unpacking in the adaptor.

— Ensure `const` and non-`const` overloads for `begin` and `end` in all views.

— Move `null_sentinel_t` to `std`, remove its `base` member function, and make it useful for more than just pointers, based on SG-9 guidance.

2 Motivation

Unicode is important to many, many users in everyday software. It is not exotic or weird. Well, it's weird, but it's not weird to see it used. C and C++ are the only major production languages with essentially no support for Unicode.

Let's fix.

To fix, first we start with the most basic representations of strings in Unicode: UTF. You might get a UTF string from anywhere; on Windows you often get them from the OS, in UTF-16. In web-adjacent applications, strings are most commonly in UTF-8. In ASCII-only applications, everything is in UTF-8, by its definition as a superset of ASCII.

Often, an application needs to switch between UTFs: 8 -> 16, 32 -> 16, etc. In SG-16 we've taken to calling such UTF-N -> UTF-M operations "transcoding".

I'm proposing interfaces to do transcoding that meet certain design requirements that I think are important; I hope you'll agree:

— Ranges are the future. We should have range-friendly ways of doing transcoding. This includes support for sentinels and lazy views.

— Iterators are the present. We should support generic programming, whether it is done in terms of pointers, a particular iterator, or an iterator type specified as a template parameter.

— A null-terminated string should not be treated as a special case. The ubiquity of such strings means that they should be treated as first-class strings.

— It is common to want to view the same text as code points and code units at different times. It is therefore important that transcoding iterators have a convenient way to access the underlying sequence of code units being transcoded.

— Memory safety is important. Ensuring that the Unicode part of the standard library is as memory safe as possible should be a priority.

§ 2.1 A note about P1629

[P1629R1] from JeanHeyd Meneide is a much more ambitious proposal that aims to standardize a general-purpose text encoding conversion mechanism. This proposal is not at odds with P1629; the two proposals have largely orthogonal aims. This proposal only concerns itself with UTF interconversions, which is all that is required for Unicode support. P1629 is concerned with those conversions, plus a lot more. Accepting both proposals would not cause problems; in fact, the APIs proposed here could be used to implement parts of the P1629 design.

There are some differences between the way that the transcode views and iterators from [P1629R1] work and the transcoding view and iterators from this paper work. First, `std::text::transcode_view` has no direct support for null-terminated strings. Second, it does not do the unpacking described in this paper. Third, it is not printable and streamable.

§ 3 The shortest Unicode primer imaginable

There are multiple encoding types defined in Unicode: UTF-8, UTF-16, and UTF-32.

A *code unit* is the lowest-level datum-type in your Unicode data. Examples are a `char8_t` in UTF-8 and a `char32_t` in UTF-32.

A *code point* is a 32-bit integral value that represents a single Unicode value. Examples are U+0041 “A” “LATIN CAPITAL LETTER A” and U+0308 “¨” “COMBINING DIAERESIS”.

A code point may be consist of multiple code units. For instance, 3 UTF-8 code units in sequence may encode a particular code point.

§ 4 A few examples

§ 4.1 Case 1: Adapt to an existing range interface taking a different UTF

In this case, we have a generic range interface to transcode into, so we use a transcoding view.

```
// A generic function that accepts sequences of UTF-16.
template<std::uc::utf16_range R>
void process_input(R r);
```

```
void                    process_input_again(std::uc::utf_view<std::uc::format::utf16,
        std::ranges::ref_view<std::string>> r);

std::u8string input = get_utf8_input();
auto input_utf16 = input | std::uc::as_utf16;

process_input(input_utf16);
process_input_again(input_utf16);
```

4.2 Case 2: Adapt to an existing iterator interface taking a different UTF

This time, we have a generic iterator interface we want to transcode into, so we want to use the transcoding iterators.

```
// A generic function that accepts sequences of UTF-16.
template<std::uc::utf16_iter I>
void process_input(I first, I last);

std::u8string input = get_utf8_input();

process_input(
              std::uc::utf_iterator<std::uc::format::utf8,    std::uc::format::utf16,
        std::u8string::iterator>(
        input.begin(), input.begin(), input.end()),
              std::uc::utf_iterator<std::uc::format::utf8,    std::uc::format::utf16,
        std::u8string::iterator>(
        input.begin(), input.end(), input.end()));

// Even more conveniently:
auto const utf16_view = input | std::uc::as_utf16;
process_input(utf16_view.begin(), utf16.end());
```

4.3 Case 3: Adapt a range of non-character-type values

Let's say that we want to take code points that we got from ICU, and transcode them to UTF-8. The problem is that ICU's code point type is `int`. Since `int` is not a character type, it's not deduced by `as_utf8` to be UTF-32 data.

```
// A generic function that accepts sequences of UTF-16.
template<std::uc::utf8_range R>
void process_input(R r);

std::vector<int> input = get_icu_code_points();
// This is ill-formed without the as_char32_t adaptation.
auto input_utf8 = input | std::uc::as_char32_t | std::uc::as_utf8;

process_input(input_utf8);
```

4.4 Case 4: Print the results of transcoding

Text processing is pretty useless without I/O. All of the Unicode algorithms operate on code points, and so the output of any of those algorithms will be in code points/UTF-32. It should be easy to print the results to a `std::ostream`, to a `std::wostream` on Windows, or using `std::format` and `std::print`. `utf_view` is therefore printable and streamable.

```
void double_print(char32_t const * str)
{
```

```
    auto utf8 = str | std::uc::as_utf8;
    std::print("{}", utf8);
    std::cerr << utf8;
}
```

5 Proposed design

5.1 Dependencies

This proposal depends on the existence of P2727 “std::iterator_interface”.

5.2 Add concepts that describe parameters to transcoding APIs

The macro `CODE_UNIT_CONCEPT_OPTION_2` is used below to indicate the two options for how to define `code_unit`. See below for a description of the two options.

```
namespace std::uc {

  enum class format { utf8 = 1, utf16 = 2, utf32 = 4 };

  inline constexpr format wchar-t-format = see below;       // exposition only

  template<class T, format F>
    concept code_unit = (same_as<T, char8_t> && F == format::utf8) ||
                        (same_as<T, char16_t> && F == format::utf16) ||
                        (same_as<T, char32_t> && F == format::utf32)
#if CODE_UNIT_CONCEPT_OPTION_2
                        || (same_as<T, char> && F == format::utf8)
                        || (same_as<T, wchar_t> && F == wchar-t-format)
#endif
        ;

  template<class T>
    concept utf8_code_unit = code_unit<T, format::utf8>;

  template<class T>
    concept utf16_code_unit = code_unit<T, format::utf16>;

  template<class T>
    concept utf32_code_unit = code_unit<T, format::utf32>;

  template<class T>
          concept  utf_code_unit  =  utf8_code_unit<T>  ||  utf16_code_unit<T>  ||
        utf32_code_unit<T>;

  template<class T, format F>
    concept code_unit_iter =
      input_iterator<T> && code_unit<iter_value_t<T>, F>;
  template<class T, format F>
    concept code_unit_pointer =
      is_pointer_v<T> && code_unit<iter_value_t<T>, F>;
  template<class T, format F>
    concept code_unit_range = ranges::input_range<T> &&
      code_unit<ranges::range_value_t<T>, F>;

  template<class T>
    concept utf8_iter = code_unit_iter<T, format::utf8>;
  template<class T>
    concept utf8_pointer = code_unit_pointer<T, format::utf8>;
```

```
  template<class T>
    concept utf8_range = code_unit_range<T, format::utf8>;

  template<class T>
    concept utf16_iter = code_unit_iter<T, format::utf16>;
  template<class T>
    concept utf16_pointer = code_unit_pointer<T, format::utf16>;
  template<class T>
    concept utf16_range = code_unit_range<T, format::utf16>;

  template<class T>
    concept utf32_iter = code_unit_iter<T, format::utf32>;
  template<class T>
    concept utf32_pointer = code_unit_pointer<T, format::utf32>;
  template<class T>
    concept utf32_range = code_unit_range<T, format::utf32>;

  template<class T>
    concept utf_iter = utf8_iter<T> || utf16_iter<T> || utf32_iter<T>;
  template<class T>
    concept utf_pointer = utf8_pointer<T> || utf16_pointer<T> || utf32_pointer<T>;
  template<class T>
    concept utf_range = utf8_range<T> || utf16_range<T> || utf32_range<T>;

  template<class T>
    concept utf_range_like =
      utf_range<remove_reference_t<T>> || utf_pointer<remove_reference_t<T>>;

  template<class T>
    concept utf8_input_range_like =
      (ranges::input_range<remove_reference_t<T>> && utf8_code_unit<iter_value_t<T>>)
        ||
      utf8_pointer<remove_reference_t<T>>;
  template<class T>
    concept utf16_input_range_like =
                                  (ranges::input_range<remove_reference_t<T>>      &&
        utf16_code_unit<iter_value_t<T>>) ||
      utf16_pointer<remove_reference_t<T>>;
  template<class T>
    concept utf32_input_range_like =
                                  (ranges::input_range<remove_reference_t<T>>      &&
        utf32_code_unit<iter_value_t<T>>) ||
      utf32_pointer<remove_reference_t<T>>;

  template<class T>
    concept utf_input_range_like =
                    utf8_input_range_like<T>    ||    utf16_input_range_like<T>    ||
        utf32_input_range_like<T>;

  template<class T>
    concept transcoding_error_handler =
      requires (T t, string_view msg) { { t(msg) } -> same_as<char32_t>; };

}
```

There are two options for how the `code_unit` concept is defined.

5.2.1 Code unit option 1

This is represented by `CODE_UNIT_CONCEPT_OPTION_2 == 0` in the code above. In this option, a code unit must be one of `char8_t`, `char16_t`, and `char32_t`.

5.2.2 Code unit option 2

This is represented by `CODE_UNIT_CONCEPT_OPTION_2 == 1` in the code above. In this option, a code unit must be a character type. This includes the `charN_t` character types from Option 1, plus `char` and `wchar_t`. The value of *`wchar-t-format`* is implementation defined, but must be `uc::format::utf16` or `uc::format::utf32`.

5.2.3 The impact of options 1 and 2

Here are some examples of the differences between Options 1 and 2. The `as_utfN` and `as_charN` adaptors are discussed later in this paper.

The `as_utfN` adaptors produce `utfN_views`, which do transcoding.

The `as_utfN` adaptors produce `charN_views` that are each very similar to a `transform_view` that casts each element of the adapted range to a `charN_t` value. A `charN_view` differs from the equivalent transform in that it may be a borrowed range, and that the `utfN_view` views know about the `charN_views`, and can optimize away the work that would be done by the `charN_view`. This turns `charN_view` into a no-op when nested within a `utfN_view`.

Note the use of `charN_t` below with `std::wstring`. That's there because whether you write `as_char16_t` or `as_char32_t` is implementation-dependent.

Option 1

```
using namespace std::uc;

auto v1  = u8"text" | as_utf32;  // Ok.
auto v2  = u"text"  | as_utf8;   // Ok.
auto v3  = U"text"  | as_utf16;  // Ok.

auto v4  = std::u8string(u8"text") | as_utf32;  // Ok.
auto v5  = std::u16string(u"text") | as_utf8;   // Ok.
auto v6  = std::u32string(U"text") | as_utf16;  // Ok.

auto v7  = std::string  | as_utf32; // Error; ill-formed.
auto v8  = std::wstring | as_utf8;  // Error; ill-formed.

auto v9  = std::string  | as_char8_t | as_utf32; // Ok.
auto v10 = std::wstring | as_charN_t | as_utf8;  // Ok.
```

Option 2

```
using namespace std::uc;

auto v1  = u8"text" | as_utf32;  // Ok.
auto v2  = u"text"  | as_utf8;   // Ok.
auto v3  = U"text"  | as_utf16;  // Ok.

auto v4  = std::u8string(u8"text") | as_utf32;  // Ok.
auto v5  = std::u16string(u"text") | as_utf8;   // Ok.
auto v6  = std::u32string(U"text") | as_utf16;  // Ok.

auto v7  = std::string  | as_utf32; // Ok.
auto v8  = std::wstring | as_utf8;  // Ok.

auto v9  = std::string  | as_char8_t | as_utf32; // Ok.
auto v10 = std::wstring | as_charN_t | as_utf8;  // Ok.
```

In short, Option 1 forces you to write "`| as_char8_t`" everywhere you want to use a `std::string` with the interfaces proposed in this paper.

Option 1 is supported by most of SG-16. Here is the relevant SG-16 poll:

UTF transcoding interfaces provided by the C++ standard library should operate on charN_t types, with support for other types provided by adapters, possibly with a special case for char and wchar_t when their associated literal encodings are UTF.

SF	F	N	A	SA
6	1	0	0	1

(I have chosen to ignore the "possibly with a special case for char and wchar_t when their associated literal encodings are UTF" part. Making the evaluation of a concept change based on the literal encoding seems like a flaky move to me; the literal encoding can change TU to TU.)

The feeling in SG-16 is that the `charN_t` types are designed to represent UTF encodings, and `char` is not. A `char const *` string could be in any one of dozens (hundreds?) of encodings. The addition of "`| as_char8_t`" to adapt ranges of `char` is meant to act as a lexical indicator of user intent.

I believe this decision is a mistake. I would very, very much *not* like to standardize Unicode interfaces that do not easily interoperate with `std::string`. This is my reasoning:

First, `char` and `char8_t` maintain exactly the same set of invariants – the empty set. Note that this is true even for string literals. The encoding of `u8"text"` is not necessarily UTF-8! It depends on the flags you pass to your compiler. Those flags are allowed to vary TU by TU. I have been bitten by the "`u8` does not necessarily mean UTF-8" oddity of MSVC before.

Second, "`| as_char8_t`" is a no-op when used with `utfN_view`/`utf_view`. It does not actually do anything to help you get your program's text into UTF-8 encoding, nor to detect that you have non-UTF-8 encoded text in your program.

Third, people use `std::string` a lot. They use `char` string literals a lot. They use `std::u8string` and `char8_t` string literals almost not at all. Using Github Code Search, I found 15.3M references to `std::string` and 6.7k references to `std::u8string`. Even were everyone to switch from `std::string` to `std::u8string` today, we should still have to deal with lots and lots of `char const *` strings for C API compatibility.

Finally, whether a given range of code units is properly UTF encoded may be a precondition of a given API that the user writes, but it is not a precondition of *any* API proposed in this paper, nor is it a precondition of any API I'm proposing in the papers that will follow this one.

In short, I think `"text" | std::uc::as_utf32` should "just work". Making users write `"text" | std::uc::as_char8_t | std::uc::as_utf32`, when that does not increase correctness or efficiency – and produces no different object code – seems wrongheaded to me. Users that want the extra explicitness can still write the longer version under both options. Users that do not want this explicitness should not be forced to write it.

§ 5.3 Add a null-terminated sequence sentinel

```
namespace std {
  struct null_sentinel_t {
    template<class I>
      requires requires { typename ITER_CONCEPT(I); } &&
               derived_from<ITER_CONCEPT(I), forward_iterator_tag> &&
               default_initializable<iter_value_t<I>> &&
               equality_comparable<iter_value_t<I>>
      friend constexpr auto operator==(I it, null_sentinel_t) { return *it ==
        iter_value_t<I>{}; }
```

```
  };

  inline constexpr null_sentinel_t null_sentinel;
}
```

This sentinel type matches any iterator position `it` at which `*it` is equal to a default-constructed object of type `iter_value_t<I>`. This works for null-terminated strings, but can also serve as the sentinel for any forward range terminated by a default-constructed value.

Because this type is potentially useful for lots of ranges unrelated to Unicode or text, it is in the `std` namespace, not `std::uc`.

If you're wondering why *`ITER_CONCEPT`* is used instead of directly requiring `forward_iterator<I>`, it's because the latter causes recursion in a check of `equality_comparable` within `forward_iterator`.

§ 5.4 Add the transcoding iterator template

I'm using P2727's `iterator_interface` here for simplicity.

First, the synopsis:

```
namespace std::uc {
  inline constexpr char32_t replacement_character = 0xfffd;

  struct use_replacement_character {
    constexpr char32_t operator()(string_view error_msg) const noexcept;
  };

  template<format Format>
  constexpr auto format-to-type() {                                    // exposition
        only
    if constexpr (Format == format::utf8) {
      return char8_t{};
    } else if constexpr (Format == format::utf16) {
      return char16_t{};
    } else {
      return char32_t{};
    }
  }

  template<class I>
  using format-to-type-t = decltype(format-to-type<I>());              // exposition
        only

  template<
    format FromFormat,
    format ToFormat,
    input_iterator I,
    sentinel_for<I> S = I,
    transcoding_error_handler ErrorHandler = use_replacement_character>
    requires convertible_to<iter_value_t<I>, format-to-type-t<FromFormat>>
  class utf_iterator;
}
```

Then the definitions:

```
namespace std::uc {
  template<class I>
  constexpr auto bidirectional-at-most() {    // exposition only
    if constexpr (bidirectional_iterator<I>) {
```

```
    return bidirectional_iterator_tag{};
  } else if constexpr (forward_iterator<I>) {
    return forward_iterator_tag{};
  } else if constexpr (input_iterator<I>) {
    return input_iterator_tag{};
  }
}

template<class I>
using bidirectional-at-most-t = decltype(bidirectional-at-most<I>()); // exposition
      only

template<typename I, bool SupportReverse = bidirectional_iterator<I>>
struct first-and-curr {                          // exposition only
  first-and-curr() = default;
  first-and-curr(I curr) : curr{curr} {}
  template<class I2>
    requires convertible_to<I2, I>
      first-and-curr(const first-and-curr<I2>& other) : curr{other.curr} {}

  I curr;
};
template<typename I>
struct first-and-curr<I, true> {                 // exposition only
  first-and-curr() = default;
  first-and-curr(I first, I curr) : first{first}, curr{curr} {}
  template<class I2>
    requires convertible_to<I2, I>
          first-and-curr(const  first-and-curr<I2>&  other)  :  first{other.first},
      curr{other.curr} {}

  I first;
  I curr;
};

struct use_replacement_character {
        constexpr   char32_t   operator()(string_view)   const   noexcept   {   return
      replacement_character; }
};

template<
  format FromFormat,
  format ToFormat,
  input_iterator I,
  sentinel_for<I> S,
  transcoding_error_handler ErrorHandler>
  requires convertible_to<iter_value_t<I>, format-to-type-t<FromFormat>>
class utf_iterator : public iterator_interface<
                       bidirectional-at-most<I>,
                       format-to-type-t<ToFormat>,
                       format-to-type-t<ToFormat>> {
public:
  using value_type = format-to-type-t<ToFormat>;

  constexpr utf_iterator() = default;

  constexpr utf_iterator(I first, I it, S last) requires bidirectional_iterator<I>
    : first_and_curr_{first, it}, last_(last) {
    if (curr() != last_)
      read();
  }
  constexpr utf_iterator(I it, S last) requires (!bidirectional_iterator<I>)
```

```
    : first_and_curr_{it}, last_(last) {
    if (curr() != last_)
      read();
  }

  template<class I2, class S2>
    requires convertible_to<I2, I> && convertible_to<S2, S>
        constexpr utf_iterator(const utf_iterator<FromFormat, ToFormat, I2, S2,
      ErrorHandler>& other) :
    buf_(other.buf_),
    first_and_curr_(other.first_and_curr_),
    buf_index_(other.buf_index_),
    buf_last_(other.buf_last_),
    last_(other.last_)
  {}

  constexpr I begin() const requires bidirectional_iterator<I> { return first(); }
  constexpr S end() const { return last_; }

  constexpr I base() const requires forward_iterator<I> { return curr(); }

  constexpr value_type operator*() const { return buf_[buf_index_]; }

  constexpr utf_iterator& operator++() {
    if (buf_index_ + 1 == buf_last_ && curr() != last_) {
      if constexpr (forward_iterator<I>) {
        advance(curr(), to_increment_);
      }
      if (curr() == last_)
        buf_index_ = 0;
      else
        read();
    } else if (buf_index_ + 1 <= buf_last_) {
      ++buf_index_;
    }
    return *this;
  }

  constexpr utf_iterator& operator--() requires bidirectional_iterator<I> {
    if (!buf_index_ && curr() != first())
      read_reverse();
    else if (buf_index_)
      --buf_index_;
    return *this;
  }

  friend constexpr bool operator==(utf_iterator lhs, utf_iterator rhs)
    requires forward_iterator<I> || requires (I i) { i != i; } {
    if constexpr (forward_iterator<I>) {
      return lhs.curr() == rhs.curr() && lhs.buf_index_ == rhs.buf_index_;
    } else {
      if (lhs.curr() != rhs.curr())
        return false;

      if (lhs.buf_index_ == rhs.buf_index_ &&
        lhs.buf_last_ == rhs.buf_last_) {
        return true;
      }

      return lhs.buf_index_ == lhs.buf_last_ &&
          rhs.buf_index_ == rhs.buf_last_;
    }
```

```
    }

    friend constexpr bool operator==(utf_iterator lhs, S rhs)
      if constexpr (forward_iterator<I>) {
        return lhs.curr() == rhs;
      } else {
        return lhs.curr() == rhs && lhs.buf_index_ == lhs.buf_last_;
      }
    }

    using base_type =                    // exposition only
      iterator_interface<bidirectional-at-most-t<I>, value_type, value_type>;
    using base_type::operator++;
    using base_type::operator--;

  private:
    constexpr void read();                                          // exposition
        only
    constexpr void read_reverse();                                  // exposition
        only

    constexpr I first() const requires bidirectional_iterator<I>    // exposition
        only
      { return first_and_curr_.first; }
    constexpr I& curr() { return first_and_curr_.curr; }            // exposition
        only
    constexpr I curr() const { return first_and_curr_.curr; }       // exposition
        only

    array<value_type, 4 / static_cast<int>(ToFormat)> buf_;         // exposition
        only

    first-and-curr<I> first_and_curr_;                              // exposition
        only

    uint8_t buf_index_ = 0;                                         // exposition
        only
    uint8_t buf_last_ = 0;                                          // exposition
        only
    uint8_t to_increment_ = 0;                                      // exposition
        only

    [[no_unique_address]] S last_;                                  // exposition
        only

    template<
      format FromFormat2,
      format ToFormat2,
      code_unit_iter<FromFormat2> I2,
      sentinel_for<I2> S2,
      transcoding_error_handler ErrorHandler2>
    friend class utf_iterator;
  };
}
```

`use_replacement_character` is an error handler type that can be used with `utf_iterator`. It accepts a `string_view` error message, and returns the replacement character. The user can substitute their own type here, which may throw, abort, log, etc.

`utf_iterator` is an iterator that transcodes from UTF-N to UTF-M, where N and M are each one of 8, 16, or 32. N may equal M. UTF-N to UTF-N operation invokes the error handler as appropriate, but does not change format. `utf_iterator` does its work by adapting an underlying range of code units.

Each code point `c` to be transcoded is decoded from `FromFormat` in the underlying range. `c` is then encoded to `ToFormat` into an internal buffer. If ill-formed UTF is encountered during the decoding step, `c` is whatever invoking the error handler returns; using the default error handler, this is `replacement_character`.

`utf_iterator` maintains certain invariants; the invariants differ based on whether `utf_iterator` is an input iterator.

For input iterators the invariant is: if `*this` is at the end of the range being adapted, then `curr() == last_`; otherwise, the position of `curr()` is always at the end of the current code point `c` within the range being adapted, and `buf_` contains the code units in `ToFormat` that comprise `c`.

For forward and bidirectional iterators, the invariant is: if `*this` is at the end of the range being adapted, then `curr() == last_`; otherwise, the position of `curr()` is always at the beginning of the current code point `c` within the range being adapted, and `buf_` contains the code units in `ToFormat` that comprise `c`.

When ill-formed UTF is encountered in the range being adapted, `utf_iterator` calls `ErrorHandler{}.operator()` to produce a character to represent the ill-formed sequence. The number and position of error handler invocations within the transcoded output is the same, whether the range being adapted is traversed forward or backward. The number and position of the error handler invocations should use the “substitution of maximal subparts” approach described in Chapter 3 of the Unicode standard.

Besides the constructors, no member function of `utf_iterator` has preconditions. As long as a `utf_iterator` `i` is constructed with proper arguments, all subsequent operations on `i` are memory safe. This includes decrementing a `utf_iterator` at the beginning of the range being adapted, and incrementing or dereferencing a `utf_iterator` at the end of the range being adapted.

If `FromFormat` and `ToFormat` are not each one of `format::utf8`, `format::utf16`, or `format::utf32`, the program is ill-formed.

If `input_iterator<I>` is `true`, `noexcept(ErrorHandler{}(""))` must be `true` as well; otherwise, the program is ill-formed.

The exposition-only member function `read` decodes the code point `c` as `FromFormat` starting from position `curr()` in the range being adapted (`c` may be `replacement_character`); sets `to_increment_` to the number of code units read while decoding `c`; encodes `c` as `ToFormat` into `buf_`; sets `buf_index_` to `0`; and sets `buf_last_` to the number of code units encoded into `buf_`. If `forward_iterator<I>` is `true`, `curr()` is set to the position it had before `read` was called. If an exception is thrown during a call to `read`, the call to `read` has no effect.

The exposition-only member function `read_reverse` decodes the code point `c` as `FromFormat` ending at position `curr()` in the range being adapted (`c` may be `replacement_character`); sets `to_increment_` to the number of code units read while decoding `c`; encodes `c` as `ToFormat` into `buf_`; sets `buf_last_` to the number of code units encoded into `buf_`; and sets `buf_index_` to `buf_last_ - 1`. If an exception is thrown during a call to `read_reverse`, the call to `read_reverse` has no effect.

5.4.1 Why `utf_iterator` is constrained the way it is

The template parameter `I` to `utf_iterator` is not constrained with `code_unit_iter<FromFormat>` as it was in earlier revisions of this paper. Instead, `I` must be an `input_iterator` whose value type is

convertible to *format-to-type-t*<FromFormat>. This allows two uses of utf_iterator that the previous constraint would not.

First, utf_iterator can be used to adapt an iterator whose value type is some non-character type. This is useful in general, since lots of existing Unicode-aware user code uses uint32_t for UTF-32, or short for UTF-16 or whatever. It is useful in particular because ICU uses int for its UTF-32/code point type.

Second, because of the first point, adaptations of ranges of non-character types can be made more efficient. Consider:

```
std::vector<int> code_points_from_icu = /* ... */;
auto v = code_points_from_icu | std::uc::as_char32_t | std::uc::as_utf8;
auto first = v.begin();
```

The type of first is:

```
std::uc::utf_iterator<std::uc::format::utf8,                    std::uc::format::utf32,
        std::vector<int>::iterator>
```

That is, the adapting iterator that as_char32_t uses is gone. This makes using as_char32_t more efficient, when used in conjunction with as_utfN. If utf_iterator's I were required to be a utf_iter, this optimization would not work.

5.4.2 Why utf_iterator is not a nested type within utf_view

Most users will use views most of the time. However, it can be useful to use iterators some of the time. For example, say I wanted to track some user-visible cursor within some bit of text. If I wanted to represent that cursor independently from the view within which it is found, it can be awkward to do so without an independent iterator template.

```
// This is the easy case.  We have the View right there, and can use
// ranges::iterator_t to get its iterator type.

template<typename View>
struct my_state_type
{
    View all_text_;
    std::ranges::iterator_t<View>> current_position_;
    // other state ...
};

// This one, not so much.  Since we don't have the View type, we have to make
// the type of current_position_ a template parameter, even if there's only one
// type ever in use for a given view.

template<typename Iterator>
struct my_other_state_type
{
    Iterator current_position_;
    // other state ...
};
```

Using utf_iterator allows us to write more specific code. Sometimes, generic code is more desirable; sometimes nongeneric code is more desirable.

```
struct my_other_state_type
{
```

```
            std::uc::utf_iterator<format::utf8,    format::utf32,    char    const*>
        current_position_;
    // other state ...
};
```

Further, `utf_iterator` has configurability options that do not work for `utfN_view`, like the `ErrorHandler` template parameter. This will not be used often, but some users will want it sometimes. I don't think such alternate uses are going to be common enough to justify complicating `utfN_view`; those uses belong in a lower-level interface like `utf_iterator`.

5.4.3 Optional: Add aliases for common `utf_iterator` specializations

```
namespace std::uc {
    template<
        utf8_iter I,
        std::sentinel_for<I> S = I,
        transcoding_error_handler ErrorHandler = use_replacement_character>
    using utf_8_to_16_iterator =
        utf_iterator<format::utf8, format::utf16, I, S, ErrorHandler>;
    template<
        utf16_iter I,
        std::sentinel_for<I> S = I,
        transcoding_error_handler ErrorHandler = use_replacement_character>
    using utf_16_to_8_iterator =
        utf_iterator<format::utf16, format::utf8, I, S, ErrorHandler>;

    template<
        utf8_iter I,
        std::sentinel_for<I> S = I,
        transcoding_error_handler ErrorHandler = use_replacement_character>
    using utf_8_to_32_iterator =
        utf_iterator<format::utf8, format::utf32, I, S, ErrorHandler>;
    template<
        utf32_iter I,
        std::sentinel_for<I> S = I,
        transcoding_error_handler ErrorHandler = use_replacement_character>
    using utf_32_to_8_iterator =
        utf_iterator<format::utf32, format::utf8, I, S, ErrorHandler>;

    template<
        utf16_iter I,
        std::sentinel_for<I> S = I,
        transcoding_error_handler ErrorHandler = use_replacement_character>
    using utf_16_to_32_iterator =
        utf_iterator<format::utf16, format::utf32, I, S, ErrorHandler>;
    template<
        utf32_iter I,
        std::sentinel_for<I> S = I,
        transcoding_error_handler ErrorHandler = use_replacement_character>
    using utf_32_to_16_iterator =
        utf_iterator<format::utf32, format::utf16, I, S, ErrorHandler>;
}
```

These aliases make it easier to spell `utf_iterators`. Consider `utf_8_to_32_iterator<char const *>` versus `utf_iterator<format::utf8, format::utf32, char const *>`. More importantly, they allow CTAD to work, as in `utf_8_to_32_iterator(first, it, last)`. These aliases are completely optional, of course. Let us poll.

5.4.4 Add `unpack_iterator_and_sentinel` CPO for iterator "unpacking"

```
struct no_op_repacker {
  template<class T> T operator()(T x) const { return x; }
};

template<format FormatTag, utf_iter I, sentinel_for<I> S, class Repack>
struct unpack_result {
  static constexpr format format_tag = FormatTag;

  I first;
  [[no_unique_address]] S last;
  [[no_unique_address]] Repack repack;
};

// CPO equivalent to:
template<utf_iter I, sentinel_for<I> S, class Repack = no_op_repacker>
   constexpr auto unpack_iterator_and_sentinel(I first, S last, Repack repack =
        Repack());
```

Any `utf_iterator ti` contains two iterators and a sentinel. If one were to adapt `ti` in another transcoding iterator `ti2`, one quickly encounters a problem – since for example `utf_iterator<format::utf32, format::utf16, utf_iterator<format::utf8, format::utf32, char const *>>` would be the size of 9 pointers! Further, such an iterator would do a UTF-8 to UTF-16 to UTF-32 conversion, when it could have done a direct UTF-8 to UTF-32 conversion instead.

One would obviously never write a type like the monstrosity above. However, it is quite possible to accidentally construct one in generic code. Consider:

```
using namespace std::uc;

template<format IterFormat, typename Iter>
void f(Iter it, null_sentinel_t) {
#if _MSC_VER
    // On Windows, do something with 'it' that requires UTF-16.
    utf_iterator<IterFormat, format::utf16, Iter, null_sentinel_t> it16;
    windows_function(it16, null_sentinel);
#endif

    // ... etc.
}

int main(int argc, char const * argv[]) {
        utf_iterator<format::utf8, format::utf32, char const *, null_sentinel_t>
        it(argv[1], null_sentinel);

    f<format::utf32>(it, null_sentinel);

    // ... etc.
}
```

This example is a bit contrived, since users will not create iterators directly like this very often. Users are much more likely to use the `utfN_view` views and `as_utfN` view adaptors being proposed below. The view adaptors are defined in such a way that they avoid this problem altogether. They do this by unpacking the view they are adapting before adapting it. For instance:

```
std::u8string str = u8"some text";

auto utf16_str = str | std::uc::as_utf16;

static_assert(std::same_as<
    decltype(utf16_str.begin()),
```

```
            std::uc::utf_iterator<std::uc::format::utf8,    std::uc::format::utf16,
        std::u8string::iterator>
>);

auto utf32_str = utf16_str | std::uc::as_utf32;

// Poof!  The utf_iterator<format::utf8, format::utf16 iterator disappeared!
static_assert(std::same_as<
    decltype(utf32_str.begin()),
            std::uc::utf_iterator<std::uc::format::utf8,    std::uc::format::utf32,
        std::u8string::iterator>
>);
```

The unpacking logic is used in the view adaptors, as shown above. This allows you to write `r | std::uc::as_utf32` in a generic context, without caring whether `r` is a range of UTF-8, UTF-16, or UTF-32. You also do not need to care about whether `r` is a common range or not. You also can ignore whether `r` is comprised of raw pointers, some other kind of iterator, or transcoding iterators.

This becomes especially useful in the APIs proposed in later papers that depend on this paper. In particular, APIs in subsequent papers accept any UTF-N iterator, and then transcode internally to UTF-32. However, this creates a minor problem for some algorithms. Consider this algorithm (not proposed) as an example.

```
template<input_iterator I, sentinel_for<I> S, output_iterator<char8_t> O>
  requires (utf8_code_unit<iter_value_t<I>> || utf16_code_unit<iter_value_t<I>>)
transcode_result<I, O> transcode_to_utf32(I first, S last, O out);
```

Such a transcoding algorithm is pretty similar to `std::ranges::copy`, in that you should return both the output iterator *and* the final position of the input iterator (`transcode_result` is an alias for `in_out_result`). For such interfaces, it can be difficult in the general case to form an iterator of type `I` to return to the user:

```
template<input_iterator I, sentinel_for<I> S, output_iterator<char8_t> O>
    requires (utf8_code_unit<iter_value_t<I>> || utf16_code_unit<iter_value_t<I>>)
transcode_result<I, O> transcode_to_utf32(I first, S last, O out) {
       // Get the input as UTF-32.   This may involve unpacking, so possibly
        decltype(r.begin()) != I.
    auto r = ranges::subrange(first, last) | uc::as_utf32;

    // Do transcoding.
    auto copy_result = ranges::copy(r, out);

    // Return an in_out_result.
    return result<I, O>{/* ??? */, copy_result.out};
}
```

What should we write for `/* ??? */`? That is, how do we get back from the UTF-32 iterator `r.begin()` to an `I` iterator? It's harder than it first seems; consider the case where `I` is `std::uc::utf_16_to_32_iterator<std::uc::utf_8_to_16_iterator<std::string::iterator>>`. The solution is for the unpacking algorithm to remember the structure of whatever iterator it unpacks, and then rebuild the structure when returning the result. To demonstrate, here is the implementation of `transcode_to_utf32` from Boost.Text:

```
template<std::input_iterator           I,           std::sentinel_for<I>           S,
        std::output_iterator<char32_t> O>
                        requires      (utf8_code_unit<std::iter_value_t<I>>      ||
        utf16_code_unit<std::iter_value_t<I>>)
transcode_result<I, O> transcode_to_utf32(I first, S last, O out)
{
```

```
    auto const r = boost::text::unpack_iterator_and_sentinel(first, last);
    auto unpacked = detail::transcode_to_32<false>(
        detail::tag_t<r.format_tag>, r.first, r.last, -1, out);
    return {r.repack(unpacked.in), unpacked.out};
}
```

Note the call to `r.repack`. This is an invocable created by the unpacking process itself.

If this all sounds way too complicated, it's not bad at all. Here's the unpacking/repacking implementation from Boost.Text: unpack.hpp.

`unpack_iterator_and_sentinel` is a CPO. It is intended to work with UDTs that provide their own unpacking implementation. It returns an `unpack_result`.

5.4.5 Why input iterators are not unpackable

Input iterators are messed up. They barely resemble the other iterators. For one thing, they are single-pass. This means that when a `utf_iterator` adapting an input iterator reads the next code point from the range it is adapting, it must leave the iterator at a location that is just after the current code point. It has no choice, since it cannot backtrack.

It is possible to unpack an input iterator in an entirely different way than other iterators. The unpack operation for input iterators could be to produce the underlying code unit iterator (the adapted input iterator itself), *plus* the current code point that the input iterator was just used to read.

However, this is not very much help. Consider a case in which we need to unpack a UTF-8 to UTF-32 transcoding iterator so we can form a UTF-8 to UTF-16 iterator instead. The unpack operation will produce an unpacked input transcoding iterator – the moral equivalent of `std::pair<I, char32_t>`.

What can you do with this? Well, you can try to construct a `utf_iterator<format::utf8, format::utf16, I>` from it. That would mean adding a constructor that takes an input iterator and a `char32_t`. This would also mean that any user transcoding iterator types that are usable with the `unpack_iterator_and_sentinel` CPO would also need to unpack their input iterator into an iterator/code point pair, and that those user types would also need to add this odd constructor.

This is all weird. It's also a pretty small use case. People don't use input iterators that often. Since this can always be added later, it is not being proposed right now.

5.5 Add code unit views and adaptors

```
namespace std::uc {
  template<class I>
    constexpr auto iterator-to-tag() {                                  // exposition
        only
      if constexpr (random_access_iterator<I>) {
        return random_access_iterator_tag{};
      } else if constexpr (bidirectional_iterator<I>) {
        return bidirectional_iterator_tag{};
      } else if constexpr (forward_iterator<I>) {
        return forward_iterator_tag{};
      } else {
        return input_iterator_tag{};
      }
    }

  template<class I>
```

```
  using iterator-to-tag_t = decltype(iterator-to-tag<I>());          // exposition
      only

template<bool Const, class V, class T>
class charn-projection-iterator                                      // exposition
      only
  : public proxy_iterator_interface<
      iterator-to-tag<ranges::iterator_t<maybe_const<Const, V>>>,
      T>
{
  using iterator = ranges::iterator_t<maybe_const<Const, V>>;        // exposition
      only

  friend std::iterator_interface_access;                             // exposition
      only
  iterator & base_reference() noexcept { return it_; }               // exposition
      only
  iterator base_reference() const { return it_; }                    // exposition
      only

  iterator it_ = iterator();                                         // exposition
      only

  friend charn-projection-sentinel<Const, V, T>;

public:
  constexpr charn-projection-iterator() = default;
  constexpr charn-projection-iterator(iterator it) : it_(std::move(it)) {}

  constexpr T operator*() const { return T(*it_); }
};

template<bool Const, class V, class T>
class charn-projection-sentinel                                      // exposition
      only
{
  using Base = maybe_const<Const, V>;                                // exposition
      only
  using sentinel = ranges::sentinel_t<Base>;                         // exposition
      only

  sentinel end_ = sentinel();                                        // exposition
      only

public:
  constexpr charn-projection-sentinel() = default;
  constexpr explicit charn-projection-sentinel(sentinel end) : end_(std::move(end))
      {}
    constexpr charn-projection-sentinel(charn-projection-sentinel<!Const, V, T> i)
      requires Const
    && convertible_to<ranges::sentinel_t<V>, ranges::sentinel_t<Base>>;

  constexpr sentinel base() const { return end_; }

  template<bool OtherConst>
    requires sentinel_for<sentinel, ranges::iterator_t<maybe_const<OtherConst, V>>>
      friend constexpr bool operator==(const charn-projection-iterator<OtherConst,
      V, T> & x,
                                       const charn-projection-sentinel & y)
        { return x.it_ == y.end_; }

  template<bool OtherConst>
                                         requires       sized_sentinel_for<sentinel,
      ranges::iterator_t<maybe_const<OtherConst, V>>>
```

```
      friend constexpr ranges::range_difference_t<maybe_const<OtherConst, V>>
           operator-(const charn-projection-iterator<OtherConst, V, T> & x, const
      charn-projection-sentinel & y)
          { return x.it_ - y.end_; }

  template<bool OtherConst>
                                        requires       sized_sentinel_for<sentinel,
      ranges::iterator_t<maybe_const<OtherConst, V>>>
      friend constexpr ranges::range_difference_t<maybe_const<OtherConst, V>>
           operator-(const charn-projection-sentinel & y, const charn-projection-
      iterator<OtherConst, V, T> & x)
          { return y.end_ - x.it_; }
};

template<ranges::view V>
   requires ranges::input_range<V> && convertible_to<ranges::range_reference_t<V>,
      char8_t>
class char8_view : public ranges::view_interface<char8_view<V>>
{
  V base_ = V();                                                    // exposition
      only

  template<bool Const>
   using iterator = charn-projection-iterator<Const, V, char8_t>;     // exposition
      only
  template<bool Const>
   using sentinel = charn-projection-sentinel<Const, V, char8_t>;     // exposition
      only

  template<format Format2, utf_range V2>
    requires ranges::view<V2>
  friend class utf_view;

public:
  constexpr char8_view() requires default_initializable<V> = default;
  constexpr explicit char8_view(V base) : base_(std::move(base)) {}

  constexpr V base() const & requires copy_constructible<V> { return base_; }
  constexpr V base() && { return std::move(base_); }

  constexpr iterator<false> begin() { return iterator<false>{ranges::begin(base_)};
      }
  constexpr iterator<true> begin() const requires ranges::range<const V>
    { return iterator<true>{ranges::begin(base_)}; }

  constexpr sentinel<false> end() { return sentinel<false>{ranges::end(base_)}; }
     constexpr iterator<false> end() requires ranges::common_range<V> { return
      iterator<false>{ranges::end(base_)}; }
    constexpr sentinel<true> end() const requires ranges::range<const V> { return
      sentinel<true>{ranges::end(base_)}; }
  constexpr iterator<true> end() const requires ranges::common_range<const V>
    { return iterator<true>{ranges::end(base_)}; }

        constexpr auto size() requires ranges::sized_range<V> { return
      ranges::size(base_); }
     constexpr auto size() const requires ranges::sized_range<const V> { return
      ranges::size(base_); }
};

template<ranges::view V>
   requires ranges::input_range<V> && convertible_to<ranges::range_reference_t<V>,
      char16_t>
class char16_view : public ranges::view_interface<char16_view<V>>
```

```
{
  V base_ = V();                                                    // exposition
      only

  template<bool Const>
   using iterator = charn-projection-iterator<Const, V, char16_t>;    // exposition
      only
  template<bool Const>
   using sentinel = charn-projection-sentinel<Const, V, char16_t>;    // exposition
      only

  template<format Format2, utf_range V2>
    requires ranges::view<V2>
  friend class utf_view;

public:
  constexpr char16_view() requires default_initializable<V> = default;
  constexpr explicit char16_view(V base) : base_(std::move(base)) {}

  constexpr V base() const & requires copy_constructible<V> { return base_; }
  constexpr V base() && { return std::move(base_); }

  constexpr iterator<false> begin() { return iterator<false>{ranges::begin(base_)};
      }
  constexpr iterator<true> begin() const requires ranges::range<const V>
    { return iterator<true>{ranges::begin(base_)}; }

  constexpr sentinel<false> end() { return sentinel<false>{ranges::end(base_)}; }
     constexpr iterator<false> end() requires ranges::common_range<V> { return
      iterator<false>{ranges::end(base_)}; }
    constexpr sentinel<true> end() const requires ranges::range<const V> { return
      sentinel<true>{ranges::end(base_)}; }
  constexpr iterator<true> end() const requires ranges::common_range<const V>
    { return iterator<true>{ranges::end(base_)}; }

          constexpr auto size() requires ranges::sized_range<V> { return
      ranges::size(base_); }
     constexpr auto size() const requires ranges::sized_range<const V> { return
      ranges::size(base_); }
};

template<ranges::view V>
   requires ranges::input_range<V> && convertible_to<ranges::range_reference_t<V>,
      char32_t>
class char32_view : public ranges::view_interface<char32_view<V>>
{
  V base_ = V();                                                    // exposition
      only

  template<bool Const>
   using iterator = charn-projection-iterator<Const, V, char32_t>;    // exposition
      only
  template<bool Const>
   using sentinel = charn-projection-sentinel<Const, V, char32_t>;    // exposition
      only

  template<format Format2, utf_range V2>
    requires ranges::view<V2>
  friend class utf_view;

public:
  constexpr char32_view() requires default_initializable<V> = default;
  constexpr explicit char32_view(V base) : base_(std::move(base)) {}
```

```
    constexpr V base() const & requires copy_constructible<V> { return base_; }
    constexpr V base() && { return std::move(base_); }

    constexpr iterator<false> begin() { return iterator<false>{ranges::begin(base_)};
        }
    constexpr iterator<true> begin() const requires ranges::range<const V>
      { return iterator<true>{ranges::begin(base_)}; }

    constexpr sentinel<false> end() { return sentinel<false>{ranges::end(base_)}; }
       constexpr iterator<false> end() requires ranges::common_range<V> { return
        iterator<false>{ranges::end(base_)}; }
     constexpr sentinel<true> end() const requires ranges::range<const V> { return
        sentinel<true>{ranges::end(base_)}; }
    constexpr iterator<true> end() const requires ranges::common_range<const V>
      { return iterator<true>{ranges::end(base_)}; }

           constexpr auto size() requires ranges::sized_range<V> { return
        ranges::size(base_); }
      constexpr auto size() const requires ranges::sized_range<const V> { return
        ranges::size(base_); }
  };

  template<class R>
  char8_view(R &&) -> char8_view<views::all_t<R>>;
  template<class R>
  char16_view(R &&) -> char16_view<views::all_t<R>>;
  template<class R>
  char32_view(R &&) -> char32_view<views::all_t<R>>;
}

namespace std::ranges {
  template<class V>
        inline constexpr bool enable_borrowed_range<uc::char8_view<V>> =
        enable_borrowed_range<V>;
  template<class V>
        inline constexpr bool enable_borrowed_range<uc::char16_view<V>> =
        enable_borrowed_range<V>;
  template<class V>
        inline constexpr bool enable_borrowed_range<uc::char32_view<V>> =
        enable_borrowed_range<V>;
}

namespace std::uc {
  inline constexpr unspecified as_char8_t;
  inline constexpr unspecified as_char16_t;
  inline constexpr unspecified as_char32_t;
}
```

`char8_view` produces a view of `char8_t` elements from another view. `char16_view` produces a view of `char16_t` elements from another view. `char32_view` produces a view of `char32_t` elements from another view. Let `charN_view` denote any one of the views `char8_view`, `char16_view`, and `char32_view`.

The names `as_char8_t`, `as_char16_t`, and `as_char32_t` denote range adaptor objects ([range.adaptor.object]). `as_char8_t` produces `char8_views`, `as_char16_t` produces `char16_views`, and `as_char32_t` produces `char32_views`. Let `as_charN_t` denote any one of `as_char8_t`, `as_char16_t`, and `as_char32_t`, and let `V` denote the `charN_view` associated with that object. Let `E` be an expression and let `T` be `remove_cvref_t<decltype((E))>`. Let `F` be the `format` enumerator associated with `as_charN_t`. If `decltype((E))` does not model `utf_pointer<T>` and if

`charN_view(E)` is ill-formed, `as_charN_t(E)` is ill-formed. The expression `as_charN_t(E)` is expression-equivalent to:

— If `T` is a specialization of `empty_view` ([range.empty.view]), then `empty_view<`*`format-to-type-t`*`<F>>{}`.

— Otherwise, if `is_pointer_v<T>` is `true`, then `V(ranges::subrange(E, null_sentinel))`.

— Otherwise, `V(E)`.

[Example 1:

```
char32_t chars[] = U"Unicode";
std::vector<int> v(std::ranges::begin(chars), std::ranges::end(chars));
for (char8_t c : s | std::uc::as_char32_t)
  cout << (char)c << ' '; // prints U n i c o d e
```

— end example]

§ 5.5.1 Why `as_charN_t` requires `utf_pointer`

It may seem odd that `foo | as_charN_t` is well formed if `decltype(foo)` is `std::vector<int>`, but ill-formed if `decltype(foo)` is `int *`. However, this is intentional.

If you write `std::vector<int>{/* ... */} | as_char32_t`, the result is always a view whose value type is `char32_t`. If you write:

```
int * ptr = /* ... */;
auto v = ptr | as_char32_t;
```

`v` may be a view of `char32_t` values that ends in a null terminator, or it may be an error that results in UB. Null-terminated strings are very common, but null-terminated strings of a non-character type are rare. It seems far more safe and idiomatic to restrict the pointer-adaptation case only to `utf_pointers`.

§ 5.6 Add transcoding views and adaptors

The macro `CODE_UNIT_CONCEPT_OPTION_2` is used below to indicate the two options for how to define *`format-of`*, based on the definition of `code_unit`.

```
namespace std::uc {
  template<typename T>
    constexpr format format-of() {                                    // exposition
        only
      if constexpr (same_as<T, char8_t>) {
        return format::utf8{};
      } else if constexpr (same_as<T, char16_t>) {
        return format::utf16{};
      } else if constexpr (same_as<T, char32_t>) {
        return format::utf32{};
  #if CODE_UNIT_CONCEPT_OPTION_2
      } else if constexpr (same_as<T, char>) {
        return format::utf8{};
      } else if constexpr (same_as<T, wchar_t>) {
        return wchar-t-format;
  #endif
      }
    }
```

```
template<utf_range V>
  requires ranges::view<V>
class unpacking_view : public ranges::view_interface<unpacking_view<V>> {
   V base_ = V();                                                                    //
       exposition only

public:
  constexpr unpacking_view() requires default_initializable<V> = default;
  constexpr unpacking_view(V base) : base_(std::move(base)) {}

  constexpr V base() const & requires copy_constructible<V> { return base_; }
  constexpr V base() && { return std::move(base_); }

  constexpr auto code_units() const noexcept {
          auto  unpacked  =  uc::unpack_iterator_and_sentinel(ranges::begin(base_),
       ranges::end(base_));
    return ranges::subrange(unpacked.first, unpacked.last);
  }

  constexpr auto begin() { return ranges::begin(code_units()); }
  constexpr auto begin() const { return ranges::begin(code_units()); }

  constexpr auto end() { return ranges::end(code_units()); }
  constexpr auto end() const { return ranges::end(code_units()); }
};

template<class R>
  unpacking_view(R &&) -> unpacking_view<views::all_t<R>>;

template<class T>
  constexpr bool is-charn-view = false;                                  // exposition
       only
template<class V>
  constexpr bool is-charn-view<char8_view<V>> = true;                    // exposition
       only
template<class V>
  constexpr bool is-charn-view<char16_view<V>> = true;                   // exposition
       only
template<class V>
  constexpr bool is-charn-view<char32_view<V>> = true;                   // exposition
       only

template<format Format, utf_range V>
  requires ranges::view<V>
class utf_view : public ranges::view_interface<utf_view<Format, V>> {
  V base_ = V();                                          // exposition only

  template<format FromFormat, class I, class S>
    static constexpr auto make_begin(I first, S last) {   // exposition only
      if constexpr (bidirectional_iterator<I>) {
        return utf_iterator<FromFormat, Format, I, S>{
          first, first, last};
      } else {
        return utf_iterator<FromFormat, Format, I, S>{first, last};
      }
    }
  template<format FromFormat, class I, class S>
    static constexpr auto make_end(I first, S last) {     // exposition only
      if constexpr (!same_as<I, S>) {
        return last;
      } else if constexpr (bidirectional_iterator<I>) {
        return utf_iterator<FromFormat, Format, I, S>{
```

```
          first, last, last};
      } else {
        return utf_iterator<FromFormat, Format, I, S>{last, last};
      }
    }

public:
  constexpr utf_view() requires default_initializable<V> = default;
  constexpr utf_view(V base) : base_{std::move(base)} {}

  constexpr V base() const & requires copy_constructible<V> { return base_; }
  constexpr V base() && { return std::move(base_); }

  constexpr auto begin() {
    constexpr format from_format = format-of<ranges::range_value_t<V>>();
    if constexpr(is-charn-view<V>) {
                      return   make_begin<from_format>(base_.impl_.begin().base(),
      base_.impl_.end().base());
    } else {
      return make_begin<from_format>(ranges::begin(base_), ranges::end(base_));
    }
  }
  constexpr auto begin() const {
    constexpr format from_format = format-of<ranges::range_value_t<const V>>();
    if constexpr(is-charn-view<V>) {
                     return   make_begin<from_format>(ranges::begin(base_.base()),
      ranges::end(base_.base()));
    } else {
      return make_begin<from_format>(ranges::begin(base_), ranges::end(base_));
    }
  }

  constexpr auto end() {
    constexpr format from_format = format-of<ranges::range_value_t<V>>();
    if constexpr(is-charn-view<V>) {
                       return   make_end<from_format>(base_.impl_.begin().base(),
      base_.impl_.end().base());
    } else {
      return make_end<from_format>(ranges::begin(base_), ranges::end(base_));
    }
  }
  constexpr auto end() const {
    constexpr format from_format = format-of<ranges::range_value_t<const V>>();
    if constexpr(is-charn-view<V>) {
                     return   make_end<from_format>(ranges::begin(base_.base()),
      ranges::end(base_.base()));
    } else {
      return make_end<from_format>(ranges::begin(base_), ranges::end(base_));
    }
  }

  friend ostream & operator<<(ostream & os, utf_view v);
  friend wostream & operator<<(wostream & os, utf_view v);
};

template<utf_range V>
  requires ranges::view<V>
class utf8_view : public utf_view<format::utf8, V> {
public:
  constexpr utf8_view() requires default_initializable<V> = default;
  constexpr utf8_view(V base) : utf_view<format::utf8, V>{std::move(base)} {}
};
```

```
  template<utf_range V>
    requires ranges::view<V>
  class utf16_view : public utf_view<format::utf16, V> {
  public:
    constexpr utf16_view() requires default_initializable<V> = default;
    constexpr utf16_view(V base) : utf_view<format::utf16, V>{std::move(base)} {}
  };
  template<utf_range V>
    requires ranges::view<V>
  class utf32_view : public utf_view<format::utf32, V> {
  public:
    constexpr utf32_view() requires default_initializable<V> = default;
    constexpr utf32_view(V base) : utf_view<format::utf32, V>{std::move(base)} {}
  };

  template<class R>
  utf8_view(R &&) -> utf8_view<views::all_t<R>>;
  template<class R>
  utf16_view(R &&) -> utf16_view<views::all_t<R>>;
  template<class R>
  utf32_view(R &&) -> utf32_view<views::all_t<R>>;
}

namespace std::ranges {
  template<uc::format Format, class V>
      inline   constexpr   bool   enable_borrowed_range<uc::utf_view<Format,   V>>   =
        enable_borrowed_range<V>;
  template<class V>
        inline    constexpr    bool    enable_borrowed_range<uc::utf8_view<V>>    =
        enable_borrowed_range<V>;
  template<class V>
        inline    constexpr    bool    enable_borrowed_range<uc::utf16_view<V>>    =
        enable_borrowed_range<V>;
  template<class V>
        inline    constexpr    bool    enable_borrowed_range<uc::utf32_view<V>>    =
        enable_borrowed_range<V>;
}

namespace std::uc {
  template<class R>
    constexpr decltype(auto) unpack-range(R && r) {
      using T = remove_cvref_t<R>;
      if constexpr (ranges::forward_range<T>) {
                auto  unpacked  =  uc::unpack_iterator_and_sentinel(ranges::begin(r),
        ranges::end(r));
        if constexpr (is_bounded_array_v<T>) {
          constexpr auto n = extent_v<T>;
          if (n && !r[n - 1])
            --unpacked.last;
          return ranges::subrange(unpacked.first, unpacked.last);
                      }   else   if   constexpr   (!same_as<decltype(unpacked.first),
        ranges::iterator_t<R>> ||
                                                  !same_as<decltype(unpacked.last),
        ranges::sentinel_t<R>>) {
          return unpacking_view(forward<R>(r));
        } else {
          return forward<R>(r);
        }
      } else {
        return forward<R>(r);
      }
    }
```

```
  inline constexpr unspecified as_utf8;
  inline constexpr unspecified as_utf16;
  inline constexpr unspecified as_utf32;
}
```

`unpacking_view` knows how to unpack a range into code unit iterators using `unpack_iterator_and_sentinel`.

`utf_view` produces a view in UTF format `Format` of the elements from another UTF view. `utf8_view` produces a UTF-8 view of the elements from another UTF view. `utf16_view` produces a UTF-16 view of the elements from another UTF view. `utf32_view` produces a UTF-32 view of the elements from another UTF view. Let `utfN_view` denote any one of the views `utf8_view`, `utf16_view`, and `utf32_view`.

The names `as_utf8`, `as_utf16`, and `as_utf32` denote range adaptor objects ([range.adaptor.object]). `as_utf8` produces `utf8_views`, `as_utf16` produces `utf16_views`, and `as_utf32` produces `utf32_views`. Let `as_utfN` denote any one of `as_utf8`, `as_utf16`, and `as_utf32`, and let `V` denote the `utfN_view` associated with that object. Let `charN_view` denote any one of `char8_view`, `char16_view`, and `char32_view`. Let `E` be an expression and let `T` be `remove_cvref_t<decltype((E))>`. Let `F` be the `format` enumerator associated with `as_utfN`. If `decltype((E))` does not model `utf_range_like`, `as_utfN(E)` is ill-formed. The expression `as_utfN(E)` is expression-equivalent to:

— If `T` is a specialization of `empty_view` ([range.empty.view]), then `empty_view<`*`format-to-type-t`*`<F>>{}`.

— Otherwise, if `T` is a specialization of `utfN_view`, then `V(E.base())`.

— Otherwise, if `T` is a specialization of `charN_view`, then `V(E)`.

— Otherwise, if `is_pointer_v<T>` is `true`, then `V(ranges::subrange(E, null_sentinel))`.

— Otherwise, `V(`*`unpack-range`*`(E))`.

[Example 1:

```
std::u32string s = U"Unicode";
for (char8_t c : s | std::uc::as_utf8)
  cout << (char)c << ' '; // prints U n i c o d e
```

— end example]

[Example 2:

```
auto * s = L"is weird.";
for (char8_t c : s | std::uc::as_utf8)
  cout << (char)c << ' '; // prints i s   w e i r d .
```

— end example]

The `ostream` and `wostream` stream operators transcode the `utf_view` to UTF-8 and UTF-16, respectively (if transcoding is needed). The `wostream` overload is only defined on Windows.

5.6.1 Why there are three `utfN_views` views plus `utf_view`

The views in `std::ranges` are constrained to accept only `std::ranges::view` template parameters. However, they accept `std::ranges::viewable_ranges` in practice, because they each have a

deduction guide that likes like this:

```
template<class R>
utf8_view(R &&) -> utf8_view<views::all_t<R>>;
```

It's not possible to make this work for `utf_view`, since to use it you must supply a `format` NTTP. So, we need the `utfN_views`. It might be possible to make `utf_view` an exposition-only implementation detail, but I think some users might find use for it, especially in generic contexts. For instance:

```
template<std::uc::format F, typename V>
auto f(std::uc::utf_view<F, V> const & view) {
    // Use F, V, and view here....
}
```

5.6.2 `unpacking_view`

For a particular `V` being adapted by `as_utfN`, there are two cases: 1) `V` is unpackable (taht is, unpacking produces different iterator/sentinel types than what you had before unpacking), and 2) `V` is not unpackable. The second case is easy; since `V` is already unpacked, you just construct a `utfN_view` from `V`, and you're done. The first case is a little harder. For that case, we either need to let `utfN_view` know statically that it must do some unpacking, or we must introduce yet another view that does it for us. Introducing a view is the right answer, because introducing an NTTP to `utfN_view` would be unergonomic. For instance:

```
template<typename V>
void f(std::uc::utf32_view<V, /* ??? */> const & utf32) {
    // ...
}
```

What do we write for the `/* ??? */` – the NTTP that indicates whether `V` is already unpacked or not? We have to do a nontrivial amount of work involving `V` to know what to write there.

So, we have `unpacking_view` instead. When `V` is unpackable, `as_utfN` returns a `utfN_view<unpacking_view<V>>`.

In the previous revision of this paper, the `as_utfN` adaptor unpacked the adapted range most of the time, except for the one case where it could not. That case was when `r` in `r | as_utfN` is an rvalue whose `begin()` and `end()` are unpackable. In that case, we needed a special-case type called `unpacking_owning_view` that would store `r` and unpack `r.begin()` and `r.end()`. This is not ideal, because doing the unpacking in the adaptor loses information. It loses information because the unpacked view used to construct `utfN_view` is a `ranges::subrange`, not the original range. For example, if you start with an lvalue `vector`, then keeping `unpacking_view<ref_view<vector<T>>>` means that you can get access to the `vector` itself with a chain of `base()` calls. You lose that if it's a `subrange<typename vector<T>::iterator, typename vector<T>::iterator>`.

5.6.3 Adaptor examples

```
struct my_text_type
{
    my_text_type() = default;
    my_text_type(std::u8string utf8) : utf8_(std::move(utf8)) {}

    auto begin() const {
        return std::uc::utf_8_to_32_iterator(
            utf8_.begin(), utf8_.begin(), utf8_.end());
    }
```

```
    auto end() const {
        return std::uc::utf_8_to_32_iterator(
            utf8_.begin(), utf8_.end(), utf8_.end());
    }

private:
    std::u8string utf8_;
};

static_assert(std::is_same_v<
              decltype(my_text_type(u8"text") | std::uc::as_utf16),

        std::uc::utf16_view<std::uc::unpacking_view<std::ranges::owning_view<my_text_type>>>>);

static_assert(std::is_same_v<
              decltype(u8"text" | std::uc::as_utf16),
              std::uc::utf16_view<std::ranges::subrange<const char8_t *>>>);

static_assert(std::is_same_v<
              decltype(std::u8string(u8"text") | std::uc::as_utf16),
              std::uc::utf16_view<std::ranges::owning_view<std::u8string>>>);

std::u8string const str = u8"text";

static_assert(std::is_same_v<
              decltype(str | std::uc::as_utf16),
              std::uc::utf16_view<std::ranges::ref_view<std::u8string const>>>);

static_assert(std::is_same_v<
              decltype(str.c_str() | std::uc::as_utf16),
                          std::uc::utf16_view<std::ranges::subrange<const  char8_t  *,
        std::uc::null_sentinel_t>>>);

static_assert(std::is_same_v<
              decltype(std::ranges::empty_view<int>{} | std::uc::as_char16_t),
              std::ranges::empty_view<char16_t>>);

std::u16string str2 = u"text";

static_assert(std::is_same_v<
              decltype(str2 | std::uc::as_utf16),
              std::uc::utf16_view<std::ranges::ref_view<std::u16string>>>);

static_assert(std::is_same_v<
              decltype(str2.c_str() | std::uc::as_utf16),
                         std::uc::utf16_view<std::ranges::subrange<const  char16_t  *,
        std::uc::null_sentinel_t>>>);
```

§ 5.6.4 Why `utf_view` always uses `utf_iterator`, even in UTF-N to UTF-N cases

You might expect that if `r` in `r | as_utfN` is already in UTF-N, `r | as_utfN` might just be `r`. This is not what the `as_utfN` adaptors do, though.

The adaptors each produce a view `utfv` that stores a view of type `V`, where `V` is made from the result of unpacking `r`. Further, `utfv.begin()` is always a specialization of `utf_iterator`. `utfv.end()` is also a specialization of `utf_iterator` (if `common_range<V>`), or otherwise the sentinel value for `V`.

This gives `r | as_utfN` some nice, consistent properties. With the exception of `empty_view<T>{} | as_utfN`, the following are always true:

— `r | as_utfN` produces well-formed UTF. Since the default `ErrorHandler` template parameter to `utf_iterator` `use_replacement_character` is always used, any ill-formed UTF is replaced with `replacement_character`. This is true even when the input was already UTF-N. Remember, the input could have been UTF-N but had ill-formed UTF in it.

— `r | as_utfN` has a consistent API. If `r | as_utfN` were sometimes `r`, and since `r` may be a reference to an array, you'd have to use `std::ranges::begin(r)` and `::end(r)` all the time. However, you'd probably write `r.begin()` and `r.end()`, only to one day get bitten by an array-reference `r`.

— `r | as_utfN` is formattable/printable. This means you can adapt anything that can be UTF-transcoded to do I/O in a consistent way. For example:

```
auto str0 = std::format("{}", std::u8string{});                    // Error: ill-formed!
auto str1 = std::format("{}", std::u8string{} | std::uc::as_utf8); // Ok.
```

§ 5.6.5 Add `utf_view` specialization of `formatter`

These should be added to the list of "the debug-enabled string type specializations" in [format.formatter.spec]. This allows `utf_view` and `utfN_view` to be used in `std::format()` and `std::print()`. The intention is that the formatter will transcode to UTF-8 if the formatter's `CharT` is `char`, or to UTF-16 or UTF-32 (which one is implementation defined) if the formatter's `CharT` is `wchar_t` – if transcoding is necessary at all.

```
namespace std {
  template<uc::format Format, class V, class CharT>
  struct formatter<uc::utf_view<Format, V>, CharT> {
  private:
    formatter<basic_string<CharT>, CharT> underlying_;                 // exposition only

  public:
    template<class ParseContext>
      constexpr typename ParseContext::iterator
        parse(ParseContext& ctx);

    template<class FormatContext>
      typename FormatContext::iterator
        format(const uc::utf_view<Format, V>& view, FormatContext& ctx) const;

    constexpr void set_debug_format() noexcept;
  };

  template<class V, class CharT>
                  struct         formatter<uc::utf8_view<V>,         CharT>         :
        formatter<uc::utf_view<uc::format::utf8, V>, CharT> {
    template<class FormatContext>
    auto format(const uc::utf8_view<V>& view, FormatContext& ctx) const {
      return formatter<uc::utf_view<uc::format::utf8, V>,CharT>::format(view, ctx);
    }
  };

  template<class V, class CharT>
                  struct         formatter<uc::utf16_view<V>,         CharT>         :
        formatter<uc::utf_view<uc::format::utf16, V>, CharT> {
    template<class FormatContext>
    auto format(const uc::utf16_view<V>& view, FormatContext& ctx) const {
      return formatter<uc::utf_view<uc::format::utf16, V>,CharT>::format(view, ctx);
```

```
    }
  };

  template<class V, class CharT>
                  struct         formatter<uc::utf32_view<V>,         CharT>         :
        formatter<uc::utf_view<uc::format::utf32, V>, CharT> {
    template<class FormatContext>
    auto format(const uc::utf32_view<V>& view, FormatContext& ctx) const {
      return formatter<uc::utf_view<uc::format::utf32, V>,CharT>::format(view, ctx);
    }
  };
}

template<class ParseContext>
  constexpr typename ParseContext::iterator
    parse(ParseContext& ctx);
```

Effects: Equivalent to:

```
return underlying_.parse(ctx);
```

```
template<class FormatContext>
  typename FormatContext::iterator
    format(const uc::utf_view<Format, V>& view, FormatContext& ctx) const;
```

Effects: Equivalent to:

```
auto adaptor = see below;
return underlying_.format(basic_string<CharT>(from_range, view | adaptor), ctx);
```

`adaptor` is `uc::as_utf8` if `CharT` is `char`. Otherwise, it is implementation defined whether `adaptor` is `uc::as_utf16` or `uc::as_utf32`.

```
constexpr void set_debug_format() noexcept;
```

Effects: Equivalent to:

```
underlying_.set_debug_format();
```

§ 5.7 Add a feature test macro

Add the feature test macro `__cpp_lib_unicode_transcoding`.

§ 5.8 Design notes

None of the proposed interfaces is subject to change in future versions of Unicode; each relates to the guaranteed-stable subset. Just sayin'.

None of the proposed interfaces allocates or throws, unless the user supplies a throwing `ErrorHandler` template parameter to `utf_iterator`.

The proposed interfaces allow users to choose amongst multiple convenience-vs-compatibility tradeoffs. Explicitly, they are:

— If you need compatibility with existing iterator-based algorithms (such as the standard algorithms), use the transcoding iterators.

— If you want streamability or the convenience of constructing ranges with a single `| as_utfN` adaptor use, use the transcoding views.

All the transcoding iterators allow you access to the underlying iterator via `.base()` (except when adapting an input iterator), following the convention of the iterator adaptors already in the standard.

The transcoding views are lazy, as you'd expect. They also compose with the standard view adaptors, so just transcoding at most 10 UTF-16 code units out of some UTF can be done with `foo | std::uc::as_utf16 | std::ranges::views::take(10)`.

Error handling is explicitly configurable in the transcoding iterators. This gives control to those who want to do something other than the default. The default, according to Unicode, is to produce a replacement character (`0xfffd`) in the output when broken UTF encoding is seen in the input. This is what all these interfaces do, unless you configure one of the iterators as mentioned above.

The production of replacement characters as error-handling strategy is good for memory compactness and safety. It allows us to store all our text as UTF-8 (or, less compactly, as UTF-16), and then process code points as transcoding views. If an error occurs, the transcoding views will simply produce a replacement character; there is no danger of UB.

A null-terminated pointer `p` to an 8-, 16-, or 32-bit string of code units is considered the implicit range `[p, null_sentinel)`. This makes user code much more natural; `"foo" | as_utf16`, `"foo"sv | as_utf16`, and `"foo"s | as_utf16` are roughly equivalent (though the iterator type of the resulting view may differ).

Iterators are constructed from more than one underlying iterator. To do iteration in many text-handling contexts, you need to know the beginning and the end of the range you are iterating over, just to be able to do iteration correctly. Note that this is not a safety issue, but a correctness one. For example, say we have a string `s` of UTF-8 code units that we would like to iterate over to produce UTF-32 code points. If the last code unit in `s` is `0xe0`, we should expect two more code units to follow. They are not present, though, because `0xe0` is the last code unit. Now consider how you would implement `operator++()` for an iterator `iter` that transcodes from UTF-8 to UTF-32. If you advance far enough to get the next UTF-32 code point in each call to `operator++()`, you may run off the end of `s` when you find `0xe0` and try to read two more code units. Note that it does not matter that `iter` probably comes from a range with an end-iterator or sentinel as its mate; inside `iter`'s `operator++()` this is no help. `iter` must therefore have the end-iterator or sentinel as a data member. The same logic applies to the other end of the range if `iter` is bidirectional — it must also have the iterator to the start of the underlying range as a data member. This unfortunate reality comes up over and over in the proposed iterators, not just the ones that are UTF transcoding iterators. This is why iterators in this proposal (and the ones to come) usually consist of three underlying iterators.

§ 6 Implementation experience

All the interfaces proposed here have been implemented, and re-implemented, several times over the last 5 years or so. They are part of a proposed (but not yet accepted!) Boost library, Boost.Text.

The library has hundreds of stars, though I'm not sure how many users that equates to. All of the interfaces proposed here are among the best-exercised in the library. There are comprehensive tests for all the proposed entities, and those entities are used as the foundation upon which all the other library entities are composed.

Though there are a lot of individual entities proposed here, at one time or another I have need each one of them, though maybe not in every UTF-N -> UTF-M permutation. Those transcoding permutations are there mostly for completeness. I have only ever needed UTF-8 <-> UTF->32 in any of my work

that uses Unicode. Frequent Windows users will also need to convert to and from UTF-16 sometimes, because that is the UTF that the OS APIs use.

§ 7 References

[P1629R1] JeanHeyd Meneide. 2020-03-02. Transcoding the world - Standard Text Encoding. https://wg21.link/p1629r1